

Build a Job Application Agent with Claude — Step-by-Step Guide

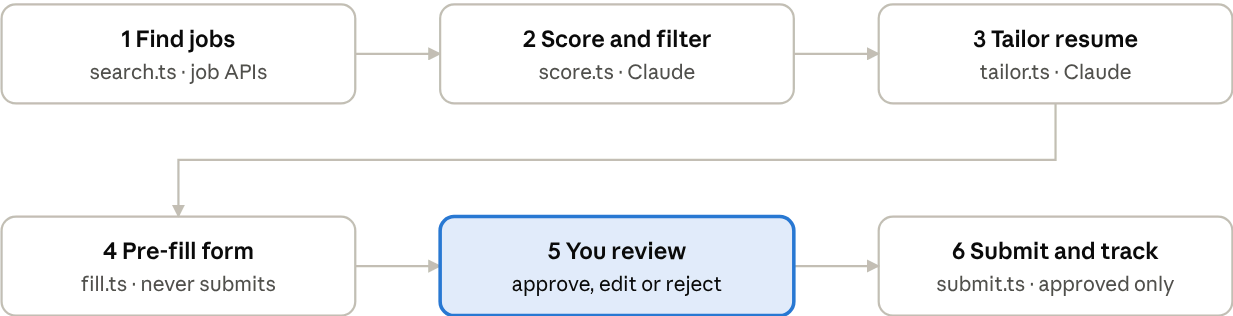
Sep 29, 2026 · @David

How the agent works

You will build a pipeline of six small programs that hand work to each other, with you as the final gate before anything is submitted. Each phase in this guide builds one piece, and each piece works on its own before you add the next.

The big idea: Claude does the thinking (matching, tailoring, answering form questions), plain code does the plumbing (fetching jobs, saving data, driving the browser), and **nothing is ever submitted without your click.**

Nothing is submitted until you approve it



job agent pipeline · 6 steps, 1 human gate

Steps 1–4 run automatically; step 5 is you, and step 6 only touches applications you approved.

Decision	Choice	Why
Language	TypeScript (Node.js)	Matches your usual stack; Playwright and the Anthropic SDK both have first-class TS support

Decision	Choice	Why
Brain	Claude via the Anthropic API (<code>@anthropic-ai/sdk</code>)	Tool use + structured JSON output is all you need; no agent framework required at first
Database	Supabase (Postgres)	You already know it; one table per stage keeps state visible
Job sources	Job APIs and public ATS job boards, not scraping LinkedIn/Indeed	Legal, stable, and far less fragile
Browser automation	Playwright, run on your own laptop	Form-filling needs a real browser; serverless hosts like Vercel can't run it well
Review UI	A small Next.js page on localhost	Where you approve, edit, or reject each application
Where it runs	Locally at first	Your resume and personal data never leave your machine except to the Claude API

Time estimate: a working version of Phases 0–4 in a weekend; Phases 5–7 in another one or two.

Before you start: four things to know

These shape the design, so read them once before writing code.

1. **LinkedIn and Indeed forbid bots.** Their terms ban automated scraping and applying, and they actively ban accounts. Use job APIs to *find* those listings, then apply on the company's own careers page (which is usually where the "Apply" button leads anyway).
2. **Company application sites vary wildly.** Greenhouse, Lever and Ashby forms are predictable and easy to automate. Workday and custom portals need logins, multi-page wizards and sometimes CAPTCHAs. Start with the easy three; for the rest, the agent prepares everything and you paste it in.
3. **Never let it invent experience.** Tailoring means re-ordering, re-wording and emphasizing what's true. A fabricated skill gets caught in the interview and can cost you the offer. The prompts in Phase 4 enforce this.

4. **Quality beats volume.** Recruiters spot mass-applied generic applications. Ten well-matched, tailored applications a week usually beat a hundred sprayed ones, so the agent filters hard before it tailors.

Phase 0: Set up your project

Goal: a TypeScript project that can call Claude and print the reply.

- ☐ **Get an Anthropic API key.** Sign up at console.anthropic.com, add a small amount of credit, and create a key. Set a monthly spend limit in the console so a bug can't run up a bill.
- ☐ **Create the project.**

```
mkdir job-agent && cd job-agent
npm init -y
npm install @anthropic-ai/sdk @supabase/supabase-js playwright zod dotenv
npm install -D typescript tsx @types/node
npx tsc --init
npx playwright install chromium
```

- ☐ **Create `.env`** (and add `.env` to `.gitignore` right away):

```
ANTHROPIC_API_KEY=sk-ant-...
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_SERVICE_KEY=...
CLAUDE_MODEL=claude-sonnet-5-5
```

- ☐ **Set up folders:**

```
job-agent/
  data/           # your resume + profile (gitignored)
  src/
    claude.ts     # one helper that talks to Claude
    profile.ts    # Phase 1
    search.ts     # Phase 2
    score.ts      # Phase 3
    tailor.ts     # Phase 4
    fill.ts       # Phase 5
  review-ui/     # Phase 6 (Next.js)
  output/        # generated resumes + screenshots
```

- ❑ **Write `src/claude.ts`**, the one helper everything else uses. It asks Claude for JSON and validates it with Zod, so bad output fails loudly instead of silently:

```
import Anthropic from "@anthropic-ai/sdk";
import { z } from "zod";
import "dotenv/config";

const client = new Anthropic();

export async function askJSON<T>(system: string, user: string, schema:
z.ZodType<T>): Promise<T> {
  const res = await client.messages.create({
    model: process.env.CLAUDE_MODEL!,
    max_tokens: 4000,
    system: system + "\nRespond with ONLY valid JSON. No prose, no code
fences.",
    messages: [{ role: "user", content: user }],
  });
  const text = res.content.map(b => (b.type === "text" ? b.text :
"")).join("");
  const clean = text.replace(/```json|```/g, "").trim();
  return schema.parse(JSON.parse(clean));
}
```

- ❑ **Test it:** write a 5-line script that calls `askJSON` with a schema like `{ joke: string }` and run it with `npx tsx src/test.ts`. When you see JSON back, Phase 0 is done.

The model name above is a placeholder you can change in one place; check the current model list in the Anthropic docs.

Phase 1: Build your profile

Goal: one `profile.json` file that holds everything true about you. Every later step reads from it, and it is the agent's only source of facts about you.

- ❑ **Save your resume as plain text** in `data/resume.md`. Markdown is easiest for Claude to read and for you to edit.
- ❑ **Write `data/preferences.json` by hand:**

```
{
  "target_titles": ["Junior Software Engineer", "Full-Stack Developer"],
  "interests": ["SaaS", "developer tools", "health tech"],
  "locations": ["Austin, TX", "Remote (US)"],
  "max_commute_miles": 30,
  "work_modes": ["remote", "hybrid"],
  "min_salary_usd": null,
  "deal_breakers": ["requires 5+ years", "unpaid", "commission only"],
  "work_authorization": "Authorized to work in the US",
  "needs_sponsorship": false
}
```

- ☐ **Write `src/profile.ts`:** send `resume.md` to Claude with a Zod schema for `basics` (name, email, phone, city, links), `education[]`, `experience[]` (company, title, dates, bullets), `projects[]`, and `skills[]`. Save the result as `data/profile.json`.
- ☐ **Read `profile.json` yourself and fix anything wrong.** This is the most important 10 minutes of the project: a mistake here gets copied into every application.
- ☐ **Add a `standard_answers` section** for questions every form asks: "Why are you interested in this role?" (a template), years of experience, earliest start date, how you heard about us, and the voluntary demographic questions ("Decline to self-identify" is always a valid choice).

Done when: `profile.json` is correct and you'd be comfortable if a recruiter read every line.

Phase 2: Find jobs

Goal: a `jobs` table in Supabase that fills with fresh, de-duplicated listings every time you run `search.ts`.

Pick your sources (start with one, add more later):

Source	What it gives you	Good for
JSearch (on RapidAPI)	Aggregated listings from Google for Jobs, incl. ones posted on LinkedIn/Indeed, with apply links	Broad search by title + location; free tier to start
Adzuna API	Search by keyword, location and radius	A second broad source; free developer key

Source	What it gives you	Good for
Greenhouse job boards	Public JSON of every opening at a company using Greenhouse	Target companies you already like
Lever postings	Same idea for companies on Lever	Target companies

Check each provider's current docs for endpoints, limits and pricing before you build.

☐ **Create the `jobs` table:**

```
create table jobs (
  id uuid primary key default gen_random_uuid(),
  source text not null,
  external_id text not null,
  title text not null,
  company text not null,
  location text,
  remote boolean,
  description text,
  apply_url text,
  ats text,                -- 'greenhouse' | 'lever' | 'ashby' | 'other'
  posted_at timestamptz,
  status text not null default 'new',
  created_at timestamptz default now(),
  unique (source, external_id)
);
```

- ☐ **Write `search.ts`**: loop over each title in `target_titles` × each place in `locations`, call the API, and map every result to the columns above. Use `upsert` with `onConflict: 'source,external_id'` so re-running never creates duplicates.
- ☐ **Detect the ATS from the apply URL.** `boards.greenhouse.io` or `job-boards.greenhouse.io` → `greenhouse`; `jobs.lever.co` → `lever`; `jobs.ashbyhq.com` → `ashby`; anything else → `other`. Phase 5 uses this.
- ☐ **Run it** and open the Supabase table editor. Done when you see 20+ real jobs near you with descriptions and apply links.

Phase 3: Score and filter jobs

Goal: Claude reads each new job, scores how well it fits you, and only the strong matches move forward. This keeps you from wasting tokens and applications on bad fits.

- ❑ **Add columns** to `jobs`: `fit_score int`, `fit_reasons jsonb`, `missing_requirements jsonb`.
- ❑ **Do cheap filters in code first.** Drop jobs whose title contains "Senior", "Staff", "Principal" or "Manager", or whose location doesn't match your list. Free, and it removes most of the noise before Claude sees anything.
- ❑ **Write `score.ts`**: for each job with `status = 'new'`, send Claude your `profile.json`, `preferences.json` and the job description, and ask for:

```
const Score = z.object({
  fit_score: z.number().int().min(0).max(100),
  reasons: z.array(z.string()),           // why it fits
  missing_requirements: z.array(z.string()), // hard requirements you don't
  meet
  deal_breaker_hit: z.boolean(),
});
```

- ❑ **Use a clear system prompt**, for example: *"You are a strict career advisor. Score fit from 0–100 based only on the candidate's profile. Treat 'required' qualifications as hard and 'nice to have' as soft. Be honest; a low score saves the candidate time."*
- ❑ **Set status:** `score ≥ 70` and no deal breaker → `shortlisted`; otherwise → `skipped`.
- ❑ **Sanity-check** 10 scores by hand. If Claude is too generous or too harsh, adjust the prompt or the threshold, not the code.

Done when: running `score.ts` turns a pile of `new` jobs into a short list you'd actually want to apply to.

Phase 4: Tailor your resume for each job

Goal: for every shortlisted job, a one-page PDF resume that emphasizes your most relevant true experience, plus a short list of what changed so you can review it fast.

- ❑ **Create an `applications` table:** `id`, `job_id`, `tailored_resume jsonb`, `changes jsonb`, `cover_letter text`, `form_answers jsonb`, `screenshot_path text`, `status text` default 'drafting'.
- ❑ **Write `tailor.ts`**. Send Claude `profile.json` + the job description and ask for the same resume structure back, re-ordered and re-worded. The system prompt is where honesty is enforced:

You tailor resumes. Rules you must never break:

1. Use **ONLY** facts in the candidate profile. Never add a skill, tool, employer, degree, date or number that is not there.
2. You may reorder sections and bullets, rewrite bullets to use the job's vocabulary when it describes the same thing, and drop less relevant items.
3. Keep it to one page (about 450 words).
4. Return a "changes" list: each edit and why it helps for this job.
5. Return "gaps": job requirements the profile cannot honestly cover.

- ☐ **Add an automatic fact-check.** In code, list every skill in the tailored resume and flag any that isn't in `profile.json`. If anything is flagged, mark the application `needs_attention` instead of moving on. A second, cheaper Claude call asking "Does resume B claim anything not supported by profile A?" also works well.
- ☐ **Render the PDF.** Make one HTML resume template (plain, single column, standard headings like Experience and Education so applicant tracking systems parse it), fill it with the tailored JSON, and have Playwright print it: `page.pdf({ path, format: 'Letter' })`. Save to `output/resumes/<company>-<job>.pdf`.
- ☐ **Optional: cover letter.** Same idea, 200 words max, only when the form asks for one.

Done when: a shortlisted job produces a clean PDF and a change list you can read in under a minute.

Phase 5: Pre-fill the application (but never submit)

Goal: a browser bot opens the application page, fills every field, uploads your tailored resume, takes a screenshot, and **stops**. The submit button is never clicked in this phase.

- ☐ **Start with Greenhouse only.** Its forms use the same field names at almost every company, so one script covers hundreds of employers. Add Lever and Ashby after it works.
- ☐ **Step 1 — read the form.** Open `apply_url` with Playwright in visible mode (`headless: false`) and collect every input: its label, type (text, select, checkbox, file), options and whether it's required.
- ☐ **Step 2 — let Claude answer.** Send Claude the field list, `profile.json` (including `standard_answers`) and the job description. Ask for `{ field_id, value, confidence, source }` per field, where `source` says which profile fact the answer came from. Tell it: *"If the profile doesn't contain the answer, return null and confidence 0. Never guess."*

- ☐ **Step 3 — fill it.** For each answer: `page.fill()` for text, `page.selectOption()` for dropdowns, `page.check()` for checkboxes, `page.setInputFiles()` for the resume PDF. Leave null answers empty.
- ☐ **Step 4 — capture and stop.** Save a full-page screenshot, store the answers in `applications.form_answers`, set status to `awaiting_review`, and close the browser *without* clicking submit.
- ☐ **Add a code-level safety lock.** In `fill.ts`, refuse to ever call `.click()` on any element whose text matches `/submit|apply|send/i`. The only code allowed to submit lives in Phase 7 and needs an approved record.
- ☐ **For "other" ATS jobs** (Workday, custom portals), skip the bot: generate the answers and resume anyway, and mark them `manual_apply` so the review screen shows copy-paste text.

If a page shows a CAPTCHA or asks you to log in or create an account, the bot stops and marks the job `manual_apply`. Those steps are yours to do.

Done when: one real Greenhouse job ends with a filled form screenshot and an unsent application.

Phase 6: The approval queue

Goal: one local web page where you see each ready application and choose Approve, Edit, or Reject. This is your "waits for my approval" step.

- ☐ **Create the app:** `npx create-next-app@latest review-ui --ts --tailwind --app`, and give it the same Supabase keys.
- ☐ **Build one page, `/queue`**, listing applications with status `awaiting_review` or `manual_apply`, newest first.
- ☐ **Each card shows:**
 - Job title, company, location, fit score and the "why it fits" reasons
 - The change list from Phase 4 and any flagged gaps, highlighted
 - The tailored resume PDF (embed it, or a link to open it)
 - The form screenshot plus every answer, with low-confidence and empty answers in yellow
 - Buttons: **Approve, Edit, Reject**
- ☐ **Edit** lets you change any form answer or resume bullet inline and saves it back to the record. After an edit, status returns to `awaiting_review` so you see the final version.

- ☐ **Approve** sets status to `approved` and records `approved_at`. **Reject** sets `rejected` with an optional reason; feed those reasons back into your Phase 3 prompt over time.
- ☐ **Run it with** `npm run dev` and open `localhost:3000/queue`. Keep it local — it shows your personal data, and you haven't added login.

Done when: you can review and approve one application in under two minutes.

Phase 7: Submit, track and follow up

Goal: approved applications get submitted, and you always know where each one stands.

- ☐ **Write** `submit.ts`. It only loads applications with `status = 'approved'`. For each one it re-opens the form, re-fills it from the saved (possibly edited) answers, takes a final screenshot, and then clicks submit.
- ☐ **Recommended for your first few weeks:** skip the auto-click. Have `submit.ts` open the filled form in a visible browser and let *you* press Submit. You'll catch form quirks the bot missed, and the extra 5 seconds per job is worth it.
- ☐ **Confirm success.** After submitting, check for a confirmation message or URL change. Found → status `submitted` with `submitted_at`. Not found → `needs_attention`, never assume it worked.
- ☐ **Track outcomes.** Add `outcome` values: `no_response`, `rejected`, `interview`, `offer`. Update them from the queue page as emails come in.
- ☐ **Follow-up reminder.** A tiny query lists applications submitted 10+ days ago with no response, so you can send a short follow-up note if you have a contact.
- ☐ **Run the pipeline in order** with one command, e.g. `npm run daily` → search → score → tailor → fill. Then open the queue with your coffee.

Done when: one application goes all the way from search to submitted, and shows up in your tracker.

Guardrails and what to build next

Keep these on the whole time:

- ☐ Nothing is submitted unless its record says `approved`, set by you in the queue.
- ☐ Tailored resumes pass the fact-check against `profile.json`.
- ☐ Unknown form answers stay empty; the bot never guesses.
- ☐ CAPTCHAs, logins and account creation are always handed to you.

- ☐ A daily cap (start with 10 applications) so quality stays high.
- ☐ API spend limit set in the Anthropic console.
- ☐ `.env`, `data/` and `output/` are in `.gitignore`.

Once the basics work, good upgrades in rough order:

1. Add Lever and Ashby form support to Phase 5.
2. Schedule `npm run daily` with cron (Mac/Linux) or Task Scheduler (Windows).
3. Use rejection reasons and interview outcomes to tune your scoring prompt.
4. Give Claude a web-search tool in Phase 4 so it can research the company for cover letters.
5. Try Claude's computer-use or browser tools for the "other" ATS portals, still stopping before submit.